Deciding What to Do? Try Planning!

Stephen Josef Wissow



University of New Hampshire

MIT-IBM Watson AI Lab

Wheeler Ruml

Masataro Asai

What is planning?

deciding what to do in order to achieve a goal

What is planning?

deciding what to do in order to achieve a goal

- core subfield of AI
- input: world model, goal definition
- output: plan or policy (how to achieve goal)

What is planning?

deciding what to do in order to achieve a goal

- core subfield of AI
- input: world model, goal definition
- output: plan or policy (how to achieve goal)

"a model-based approach to action selection"

What is planning?

deciding what to do in order to achieve a goal

- core subfield of AI
- input: world model, goal definition
- output: plan or policy (how to achieve goal)

“a model-based approach to action selection”

distinct from:

- execution
- learning

Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

What kind of problem?

world model answers four questions:

1. How does the world work, and in particular does a given action always have the same effect?

What kind of problem?

world model answers four questions:

1. How does the world work, and in particular does a given action always have the same effect?
2. How much of the state of the world can the planner observe?

What kind of problem?

world model answers four questions:

1. How does the world work, and in particular does a given action always have the same effect?
2. How much of the state of the world can the planner observe?
3. Can the world change during the planning process?

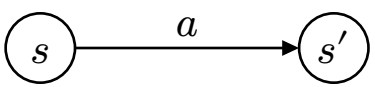
What kind of problem?

world model answers four questions:

1. How does the world work, and in particular does a given action always have the same effect?
2. How much of the state of the world can the planner observe?
3. Can the world change during the planning process?
4. Can (or must!) execution begin before planning is complete?

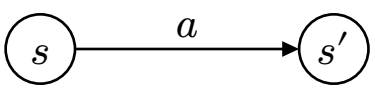
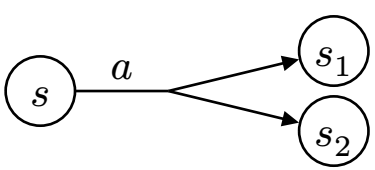
How does the world work?

question: Does a given action always have the same effect?

answer	transitions	solution	setting
yes	*deterministic* 	sequence	classical

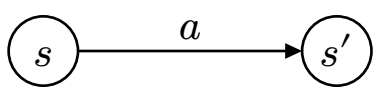
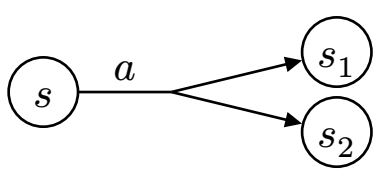
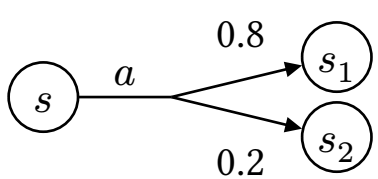
How does the world work?

question: Does a given action always have the same effect?

answer	**transitions**		**solution**	**setting**
yes	*deterministic*		sequence	classical
no	*non-deterministic*		*policy*	FOND

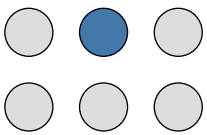
How does the world work?

question: Does a given action always have the same effect?

answer	transitions		solution	setting
yes	*deterministic*		sequence	classical
no	*non-deterministic*		*policy*	FOND
	stochastic		*policy*	MDP

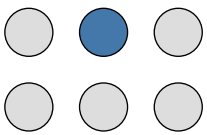
How much of the state of world can the planner see?

- *fully* observable: knows the current state
  - e.g. game of Go: whole board in view

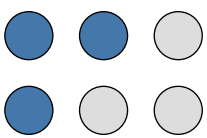


How much of the state of world can the planner see?

- *fully* observable: knows the current state
  - e.g. game of Go: whole board in view

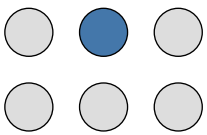


- *partially* observable: POMDP
  - e.g., doctor: disease known only through tests

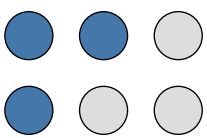


How much of the state of world can the planner see?

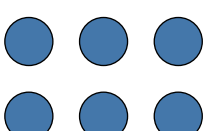
- *fully* observable: knows the current state
  - e.g. game of Go: whole board in view



- *partially* observable: POMDP
  - e.g., doctor: disease known only through tests

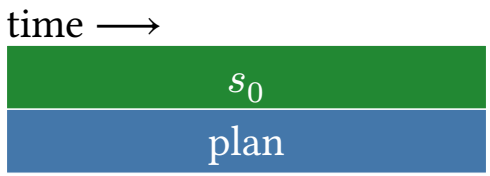


- *unobservable*: conformant
  - plan must work in any state
  - e.g. safing sequence on space station or vehicle



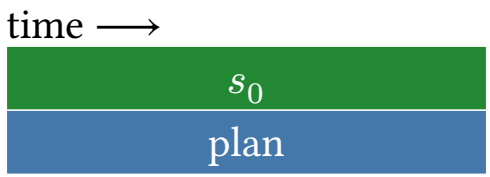
Can the world change during the planning process?

- no: *static* — only agent's actions change world

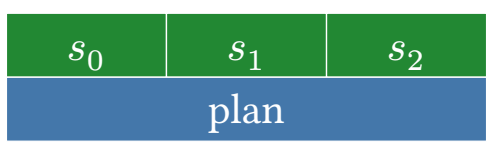


Can the world change during the planning process?

- no: *static* — only agent's actions change world



- yes: *exogenous* events, *situated* planning
  - e.g., planning how to escape approaching tiger







Can (or must) execution begin before planning is complete?

- no: *offline* — plan first, then act



Can (or must) execution begin before planning is complete?

- no: *offline* — plan first, then act



- yes: *online* — interleave planning and action
  - e.g., self-driving car keeps planning as it moves



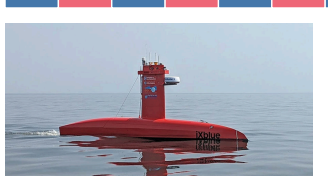
Can (or must) execution begin before planning is complete?

- no: *offline* — plan first, then act
- yes: *online* — interleave planning and action
  - e.g., self-driving car keeps planning as it moves
- must: *real-time* — commit to each move within a time limit
  - e.g., UNH CCOM's DriX









Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

Heuristic Search Viewed as Path Finding in a Graph

(Pohl 1970)

- start in *initial state* of the world



Heuristic Search Viewed as Path Finding in a Graph

(Pohl 1970)

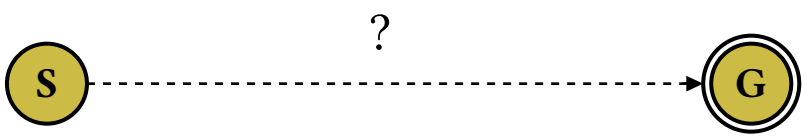
- start in *initial state* of the world
- want to achieve *goal* — e.g. eat cookie



Heuristic Search Viewed as Path Finding in a Graph

(Pohl 1970)

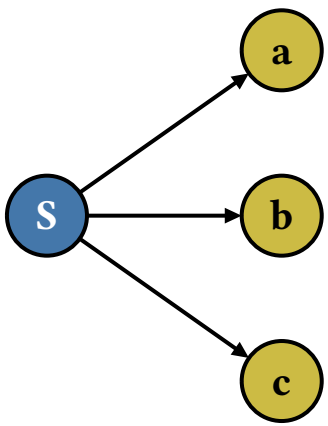
- start in *initial state* of the world
- want to achieve *goal* — e.g. eat cookie
- what actions get us there?



Heuristic Search Viewed as Path Finding in a Graph

(Pohl 1970)

- start in *initial state* of the world
- want to achieve *goal* — e.g. eat cookie
- what actions get us there?
- simulate action sequences: expand states from the *search frontier*

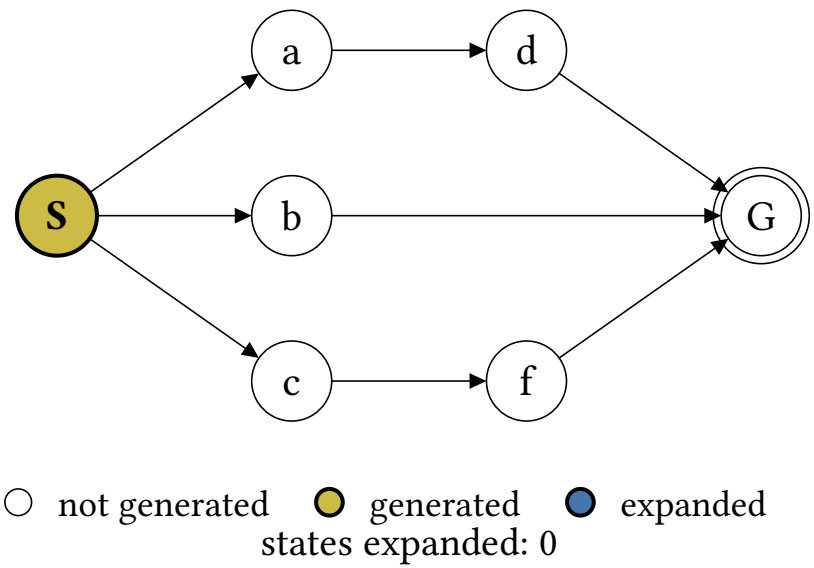


Graph search: *Depth*-First

expand newest state first

no guarantee about which path it finds

early goal detection at *generation*

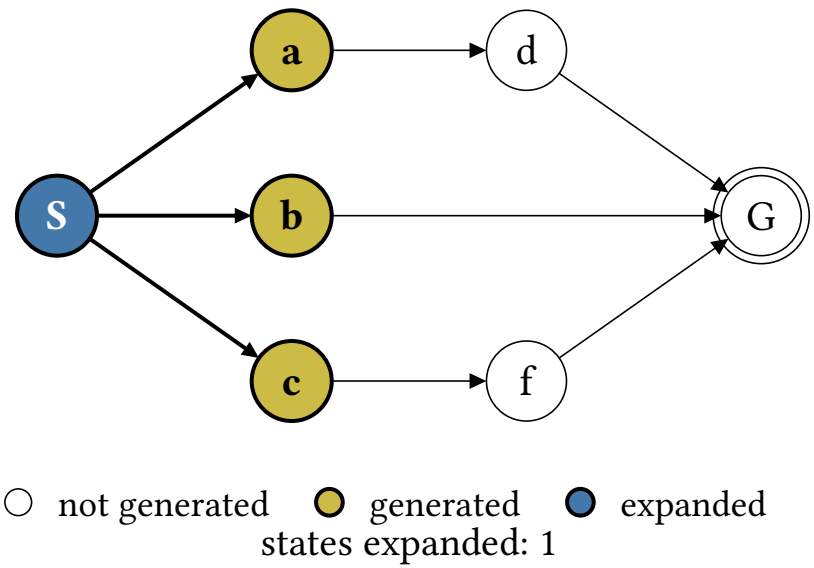


Graph search: *Depth*-First

expand newest state first

no guarantee about which path it finds

early goal detection at *generation*

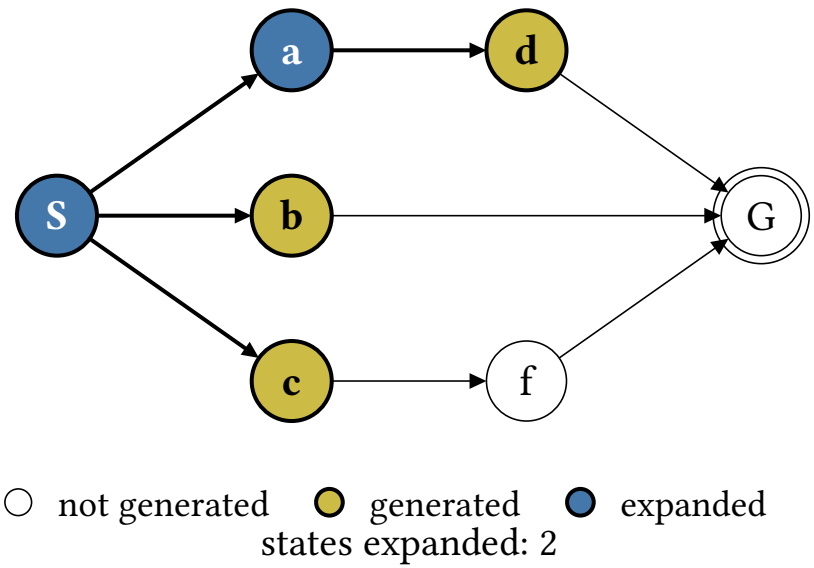


Graph search: *Depth*-First

expand newest state first

no guarantee about which path it finds

early goal detection at *generation*

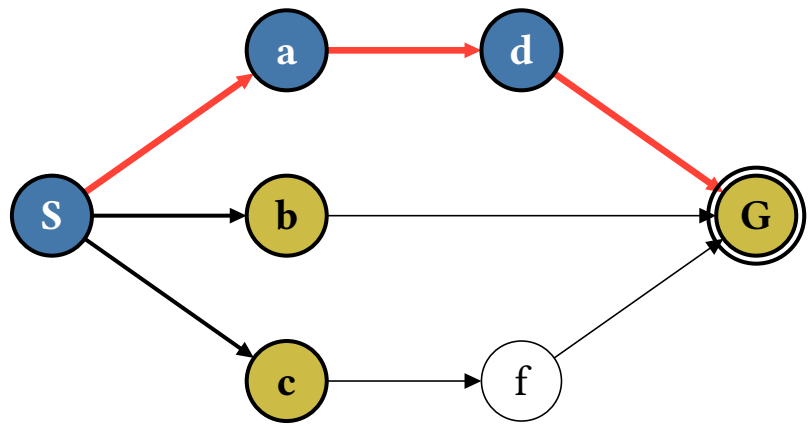


Graph search: *Depth*-First

expand newest state first

no guarantee about which path it finds

early goal detection at *generation*



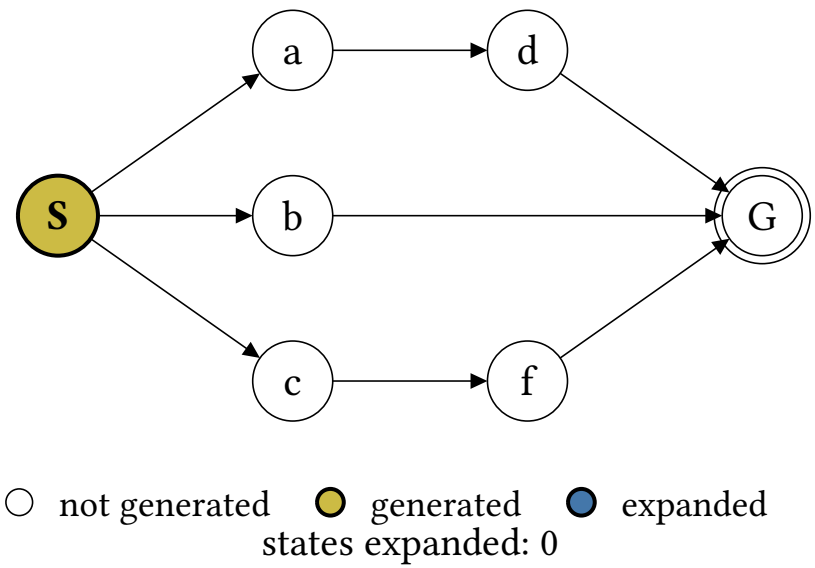
Graph search: *Breadth*-First

expand oldest state first

shortest-path: fewest actions

early goal detection

but: ignores action cost



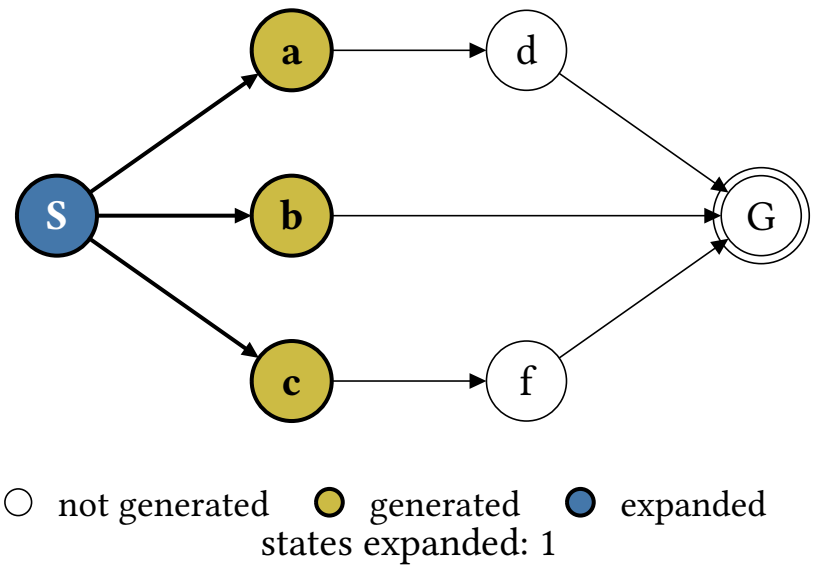
Graph search: *Breadth*-First

expand oldest state first

shortest-path: fewest actions

early goal detection

but: ignores action cost



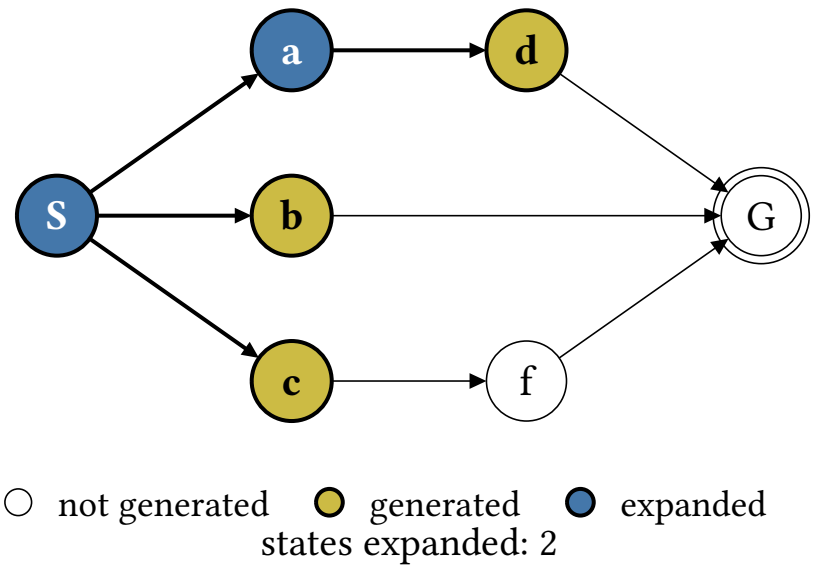
Graph search: *Breadth*-First

expand oldest state first

shortest-path: fewest actions

early goal detection

but: ignores action cost



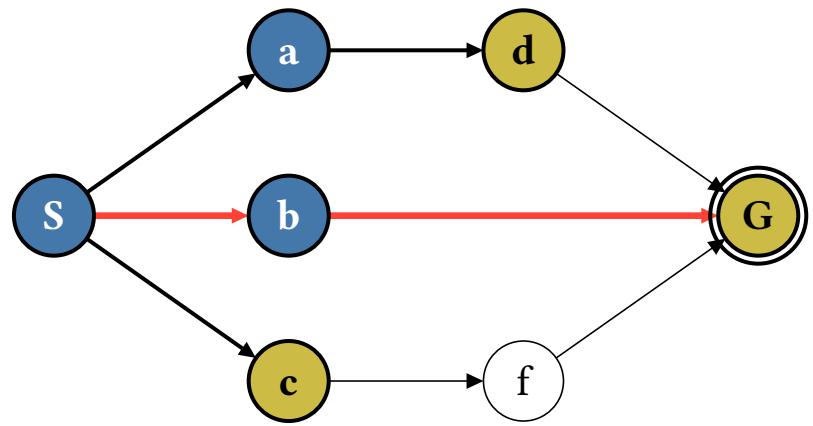
Graph search: *Breadth*-First

expand oldest state first

shortest-path: fewest actions

early goal detection

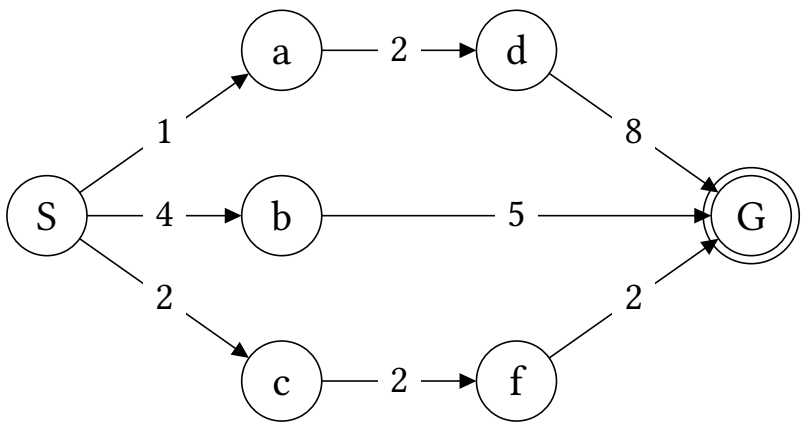
but: ignores action cost



Actions with different costs?

cost models:

- unit: every action has same cost
- non-unit: actions can have different costs



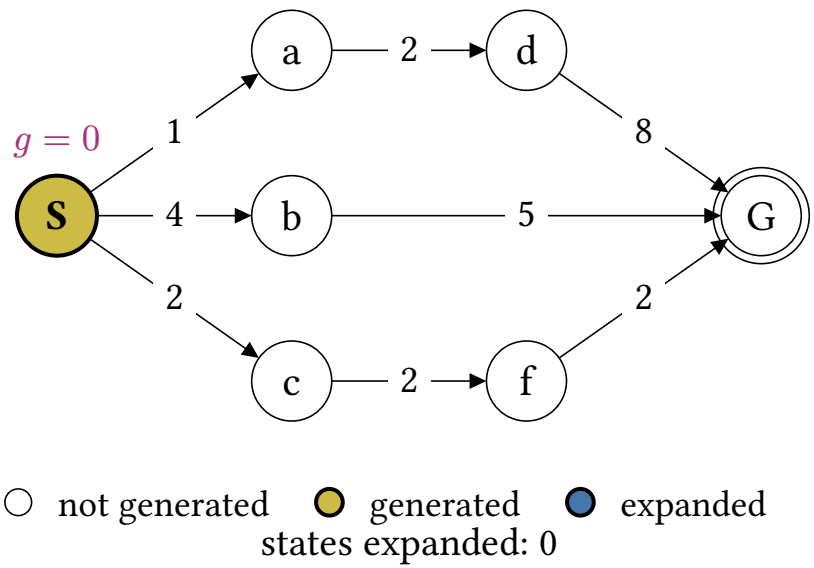
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



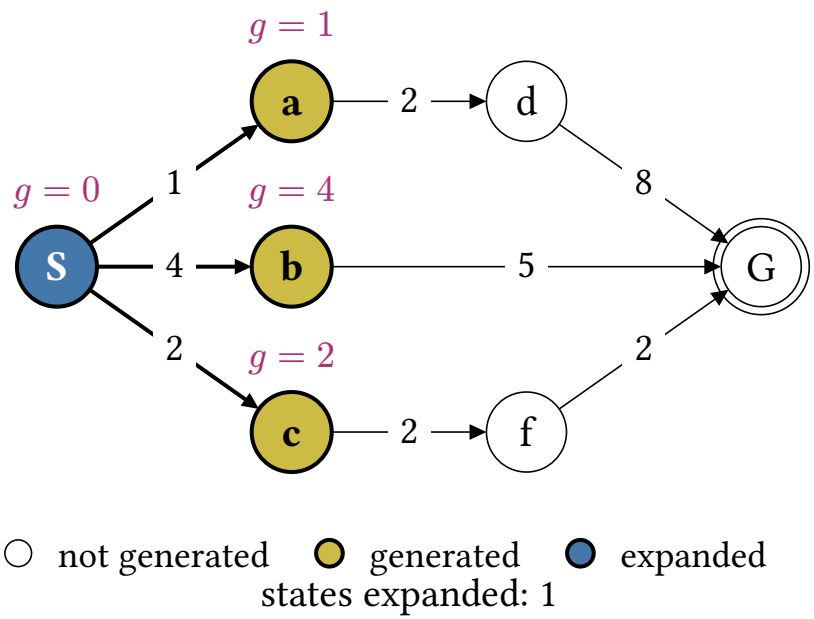
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



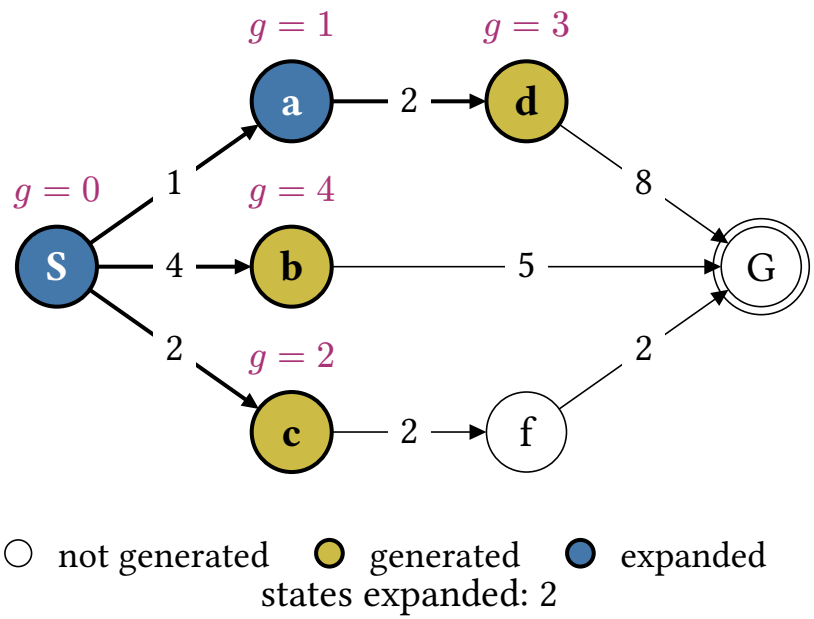
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



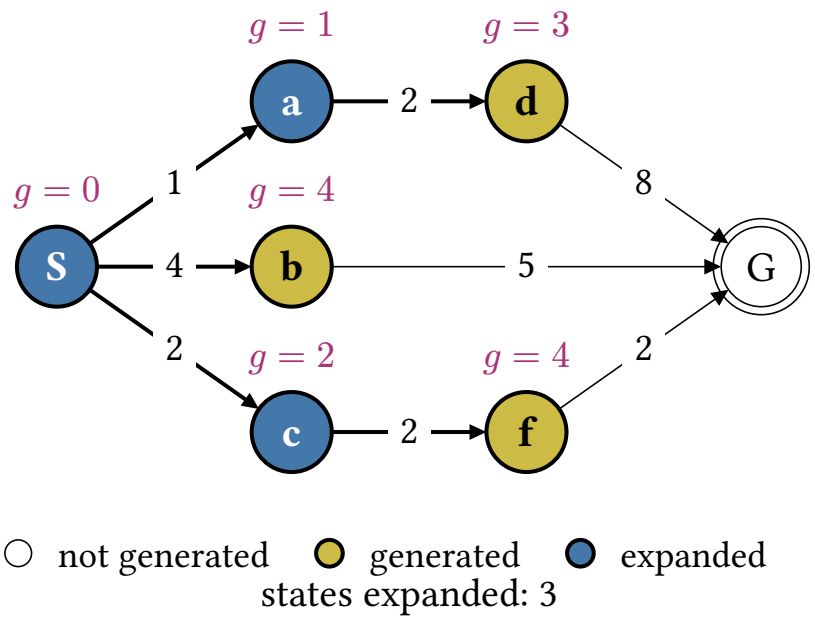
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



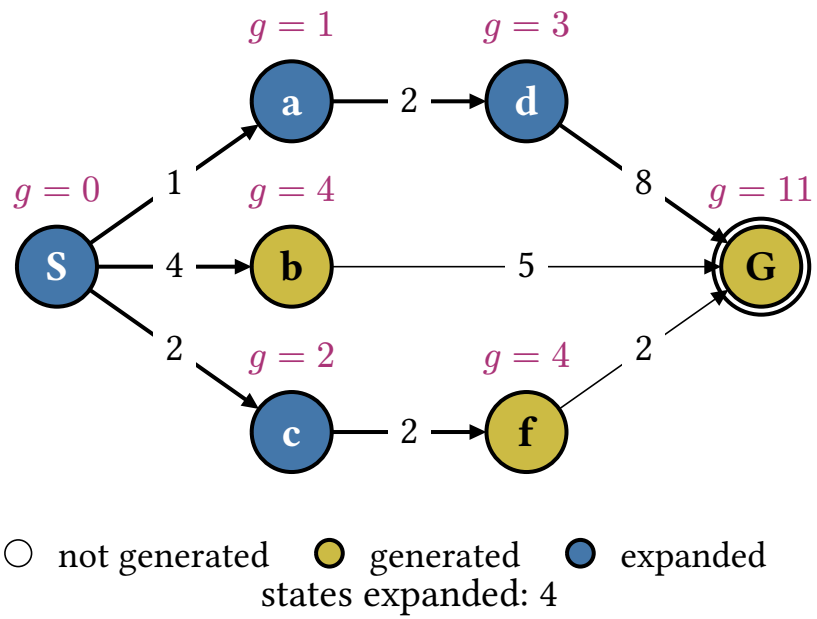
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



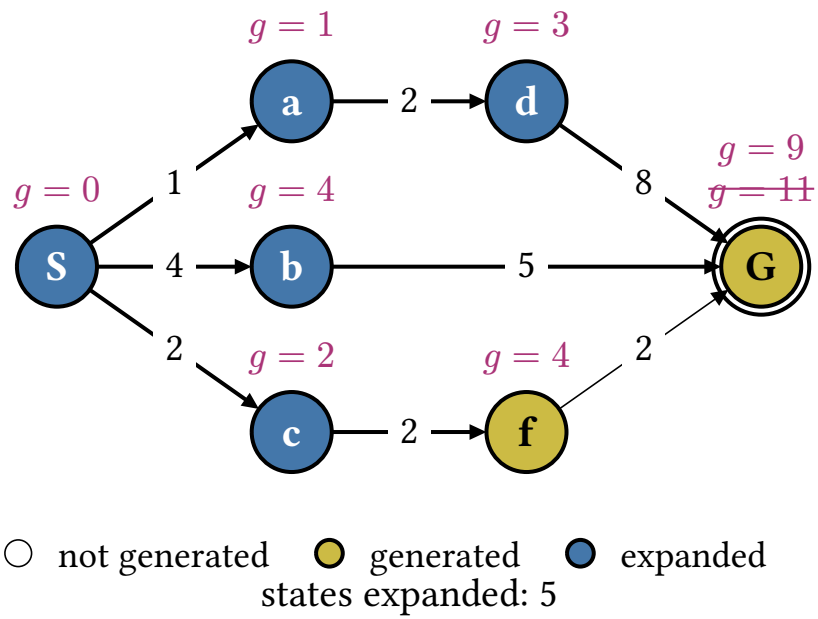
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



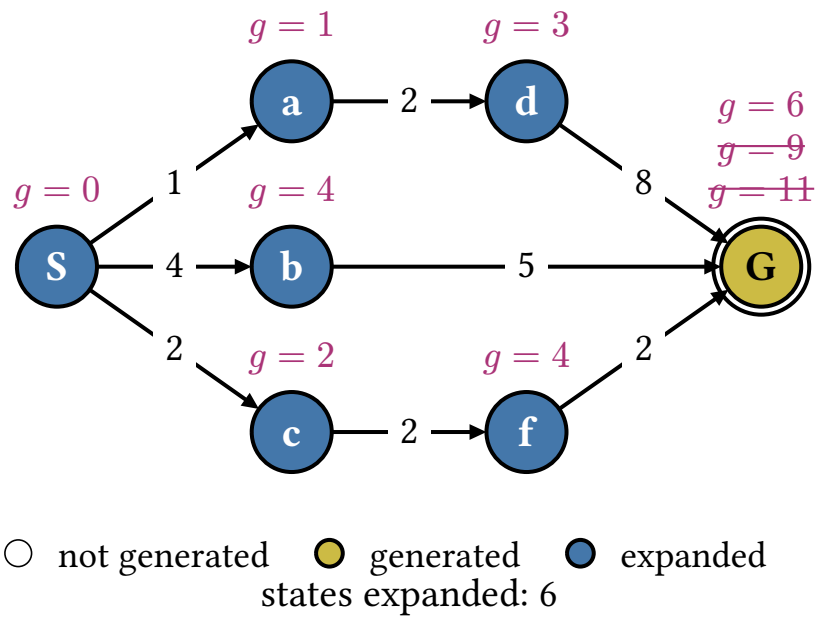
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

- terminate when goal *selected*



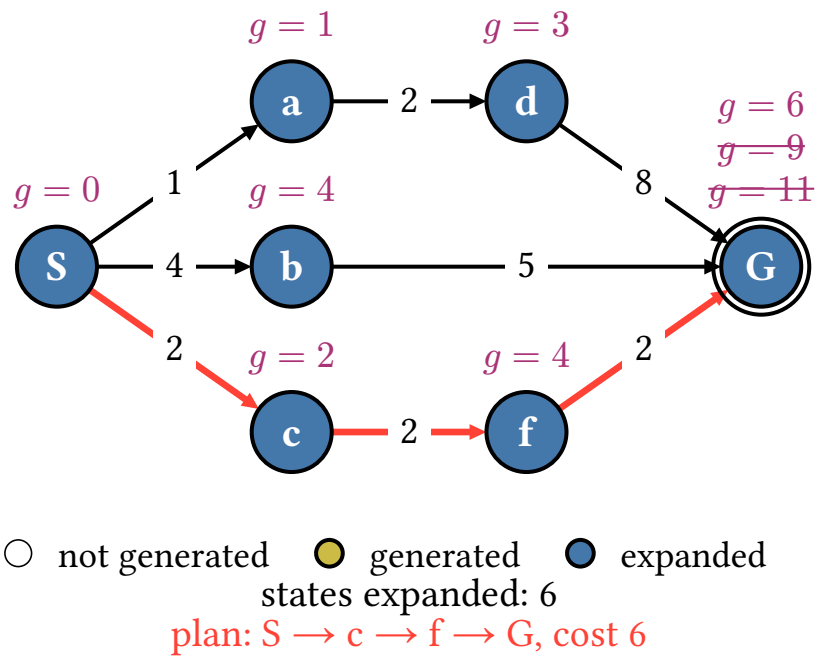
Graph search: *Best*-First

Uniform Cost Search (UCS): expand lowest *cost-so-far* — $g(s)$

- *optimal*: never misses a cheaper plan
- like Dijkstra but on infinite graphs
- frontier shaped by $g(s)$

optimality requires *late goal detection:*

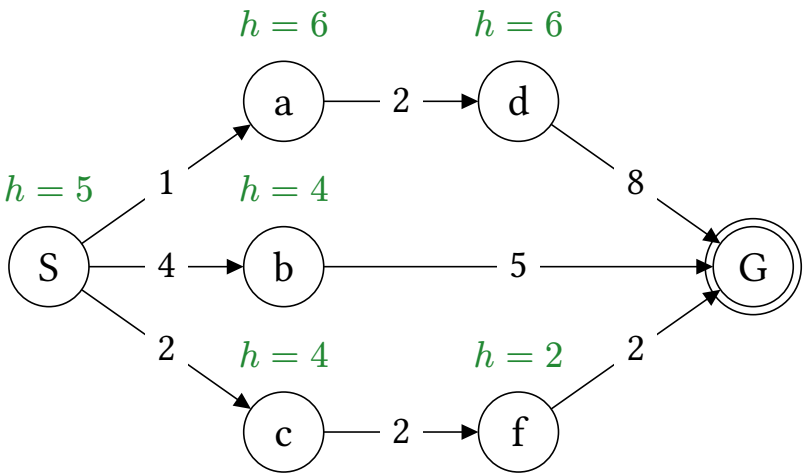
- terminate when goal *selected*



Can we do less work?

heuristic:

- like sense of smell:
  - which way next? follow your nose
- estimates remaining *cost-to-go* — $h(s)$



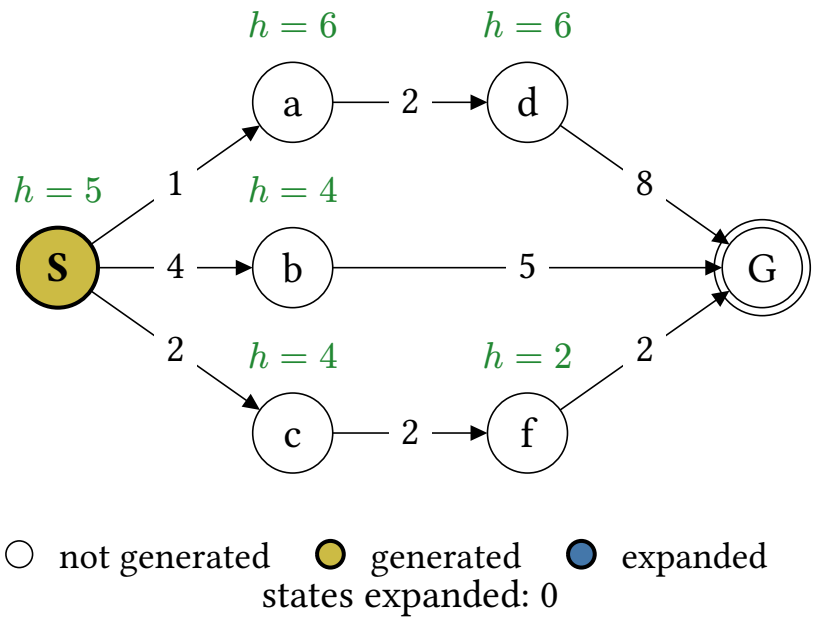
Heuristic State-Space Search

A* (Hart et al. 1968): expand the state with lowest estimated *total* cost

- $g(s)$: cost-so-far to s
- $h(s)$: estimated cost-to-go from s
- $f(s) = g(s) + h(s)$:
  - estimated total cost through s

optimality requires *admissible* h:

- never overestimate true cost-to-go



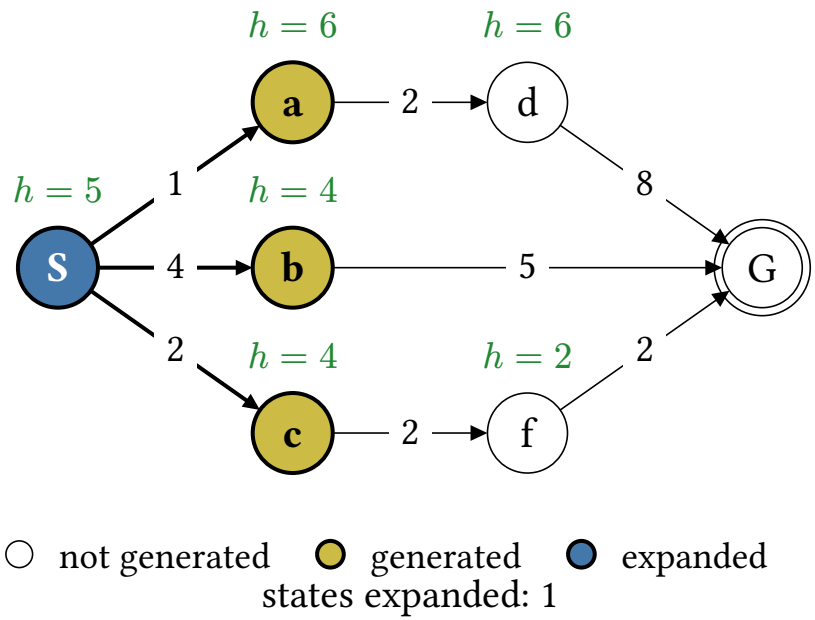
Heuristic State-Space Search

A* (Hart et al. 1968): expand the state with lowest estimated *total* cost

- $g(s)$: cost-so-far to s
- $h(s)$: estimated cost-to-go from s
- $f(s) = g(s) + h(s)$:
  - estimated total cost through s

optimality requires *admissible* h:

- never overestimate true cost-to-go



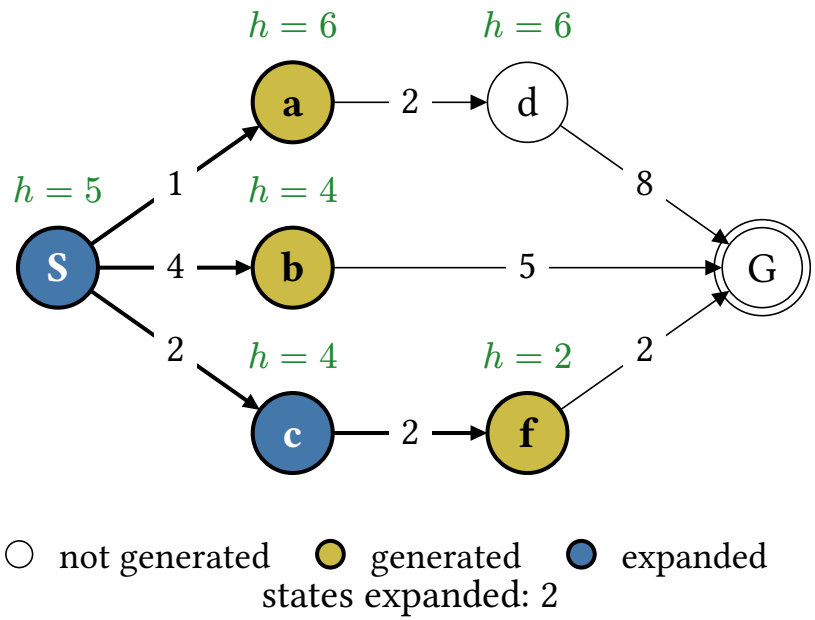
Heuristic State-Space Search

A* (Hart et al. 1968): expand the state with lowest estimated *total* cost

- $g(s)$: cost-so-far to s
- $h(s)$: estimated cost-to-go from s
- $f(s) = g(s) + h(s)$:
  - estimated total cost through s

optimality requires *admissible* h:

- never overestimate true cost-to-go



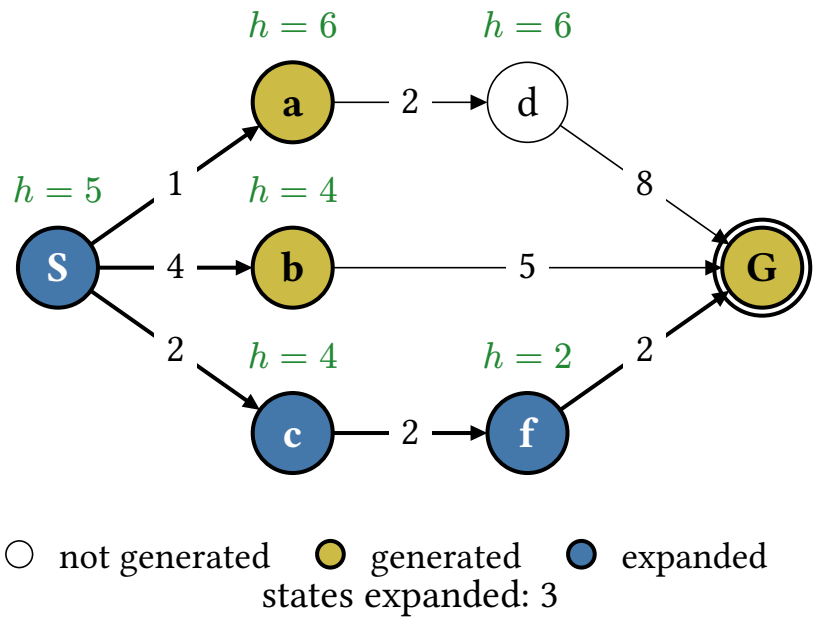
Heuristic State-Space Search

A* (Hart et al. 1968): expand the state with lowest estimated *total* cost

- $g(s)$: cost-so-far to s
- $h(s)$: estimated cost-to-go from s
- $f(s) = g(s) + h(s)$:
  - estimated total cost through s

optimality requires *admissible* h:

- never overestimate true cost-to-go



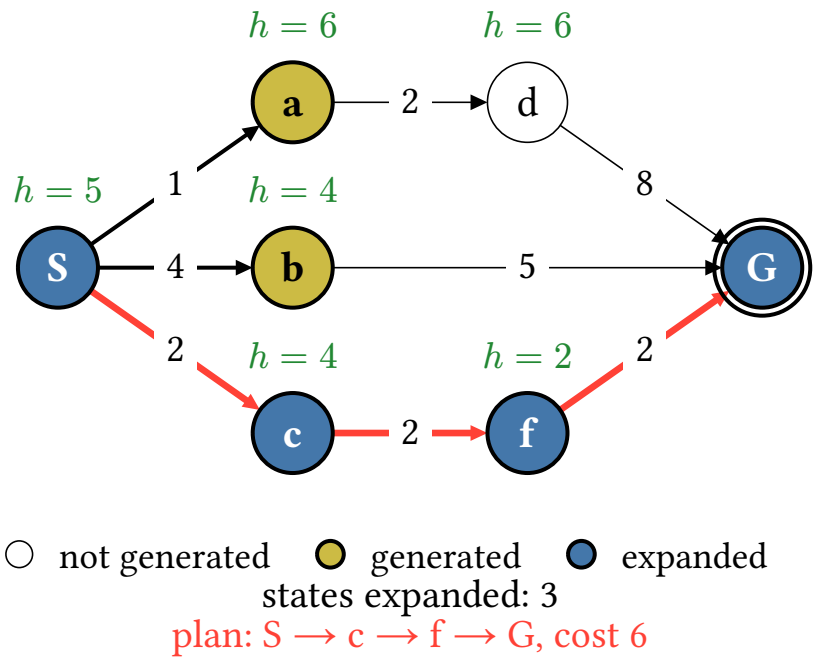
Heuristic State-Space Search

A* (Hart et al. 1968): expand the state with lowest estimated *total* cost

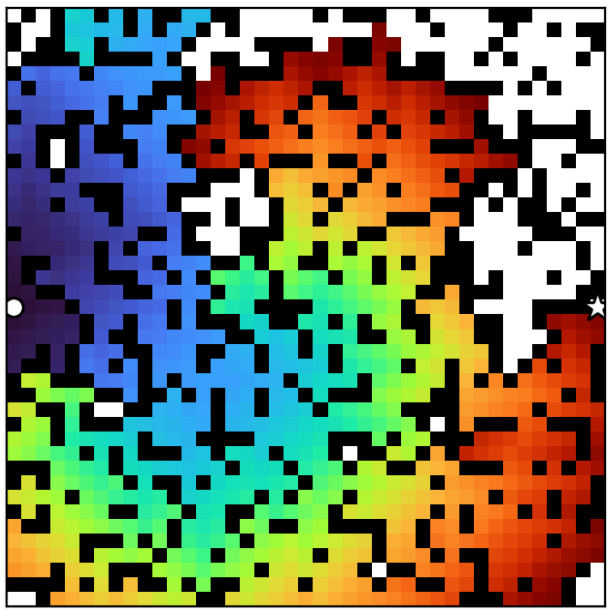
- $g(s)$: cost-so-far to s
- $h(s)$: estimated cost-to-go from s
- $f(s) = g(s) + h(s)$:
  - estimated total cost through s

optimality requires *admissible* h:

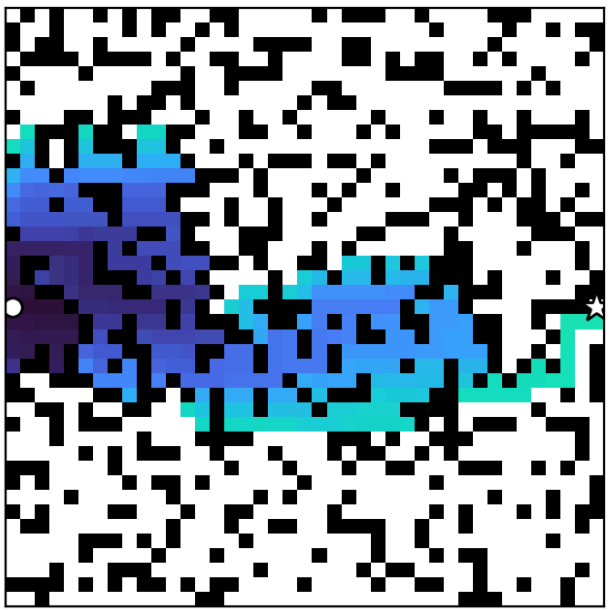
- never overestimate true cost-to-go



UCS vs A*: expansion order



UCS: 937 expansions



A*: 312 expansions

same plan cost; same color = same expansion number, blue first, red last

Astronomical? Meh.

how many quarks in universe?



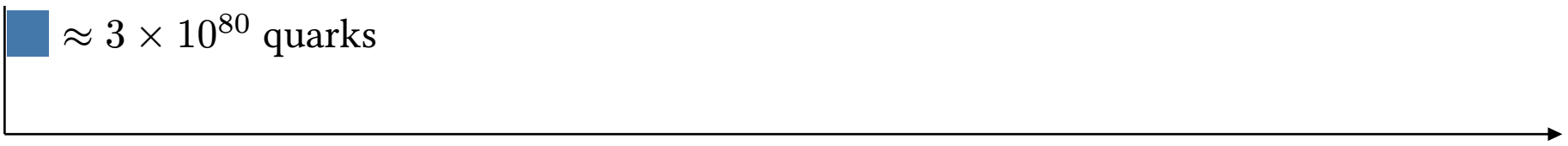
Astronomical? Meh.

how many quarks in universe?

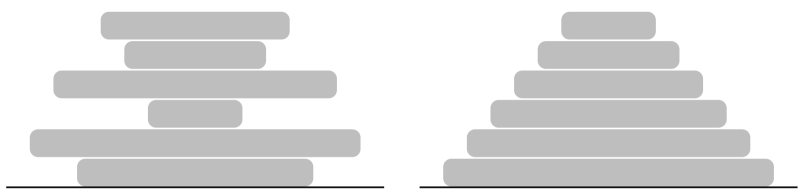


Astronomical? Meh.

how many quarks in universe?



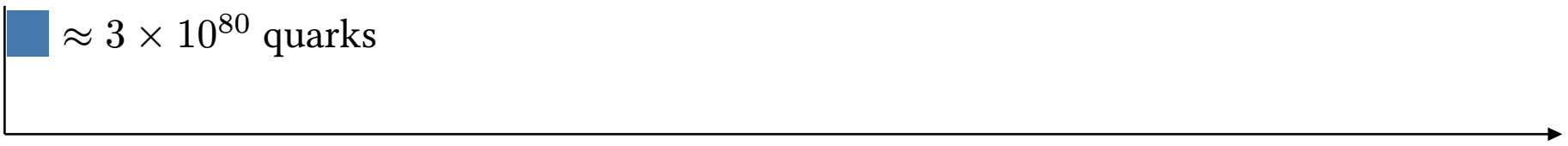
Pancakes: sort stack by flipping



(Helmert SoCS-10, Fig. 1)

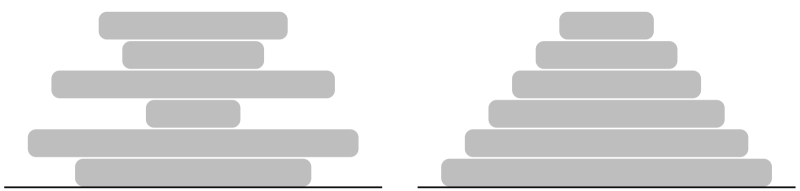
Astronomical? Meh.

how many quarks in universe?



Pancakes: sort stack by flipping

How about 60? $n = 60!$ states...



(Helmert SoCS-10, Fig. 1)

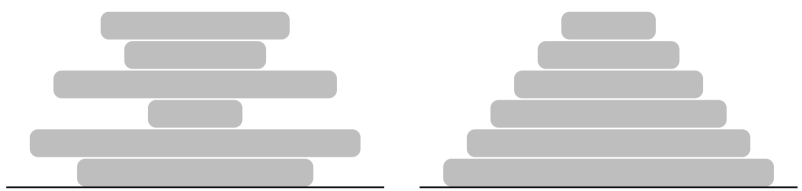
Astronomical? Meh.

how many quarks in universe?



Pancakes: sort stack by flipping

How about 60? $n = 60!$ states...



(Helmert SoCS-10, Fig. 1)

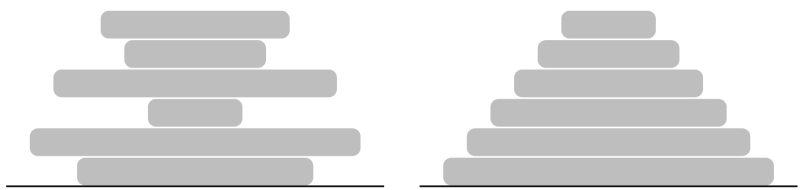
Astronomical? Meh.

how many quarks in universe?



Pancakes: sort stack by flipping

How about 60? $n = 60!$ states...



(Helmert SoCS-10, Fig. 1)

IDA* (Korf AIJ-85): **~30 seconds on 2010 hardware**, $< 10^8$ nodes

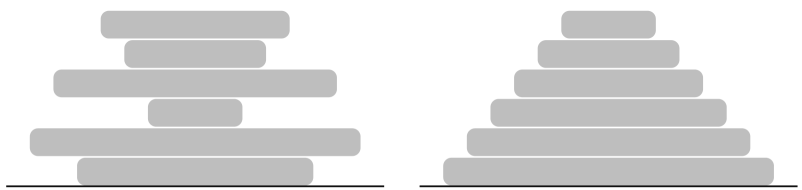
Astronomical? Meh.

how many quarks in universe?



Pancakes: sort stack by flipping

How about 60? $n = 60!$ states...



(Helmert SoCS-10, Fig. 1)

IDA* (Korf AIJ-85): **~30 seconds on 2010 hardware**, $< 10^8$ nodes

planning solves big problems efficiently

Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

Search settings: objectives and constraints

trade-offs: planning time, memory, solution cost

Search settings: objectives and constraints

trade-offs: planning time, memory, solution cost

- *optimal*: cheapest plan
- *greedy*: any plan, fast, no cost guarantee
- *bounded suboptimal*
  - objective: minimize planning time
  - constraint: solution within given constant factor of optimal

Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

Classical planning

- deterministic actions
- static world
- fully observable
- offline

Classical planning

- deterministic actions
- static world
- fully observable
- offline
- **factored state space**: state is set facts

Classical planning

- deterministic actions
- static world
- fully observable
- offline
- **factored state space**: state is set facts
- write world model in a *language*: STRIPS, PDDL, SAS, …

Classical planning: cake world

initial state: Have(Cake)

actions:

- **eat:** pre: Have(Cake); post: Eaten(Cake), ¬Have(Cake)
- **bake:** pre: ¬Have(Cake); post: Have(Cake)

goal: Have(Cake), Eaten(Cake)

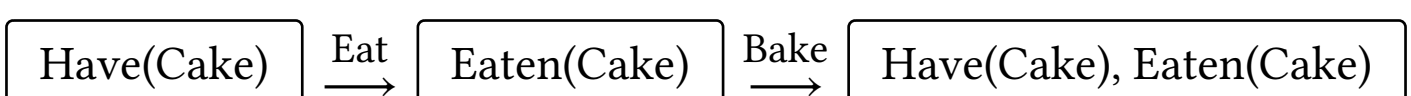
Classical planning: cake world

initial state: Have(Cake)

actions:

- **eat:** pre: Have(Cake); post: Eaten(Cake), ¬Have(Cake)
- **bake:** pre: ¬Have(Cake); post: Have(Cake)

goal: Have(Cake), Eaten(Cake)



Domain-independent heuristics

work on *any* world model expressed in the same model language

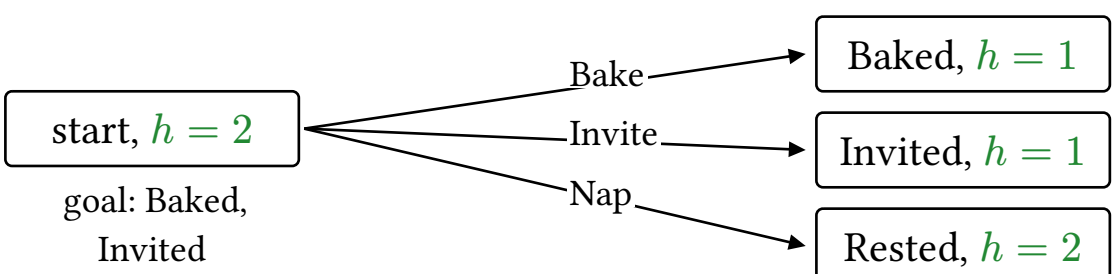
write algorithm & heuristic once, use on many problem domains

Domain-independent heuristics

work on *any* world model expressed in the same model language

write algorithm & heuristic once, use on many problem domains

pedagogical example: *goal count*, $h(s)$ = goal facts not yet true in s

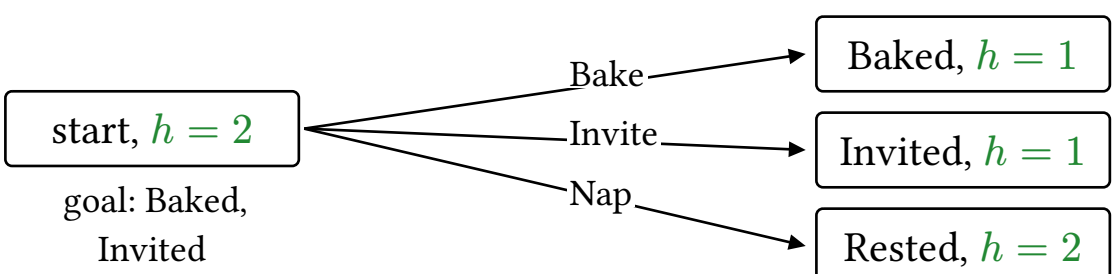


Domain-independent heuristics

work on *any* world model expressed in the same model language

write algorithm & heuristic once, use on many problem domains

pedagogical example: *goal count*, $h(s)$ = goal facts not yet true in s



lots of research into much more accurate ones: relaxations, landmarks

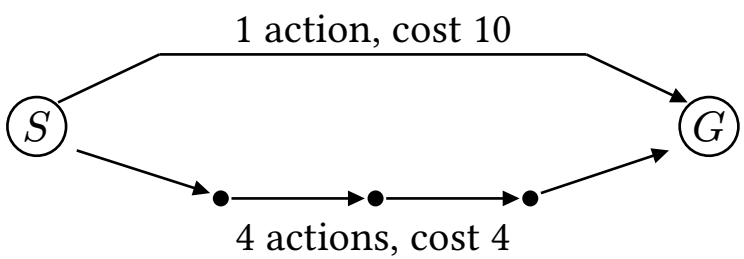
Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

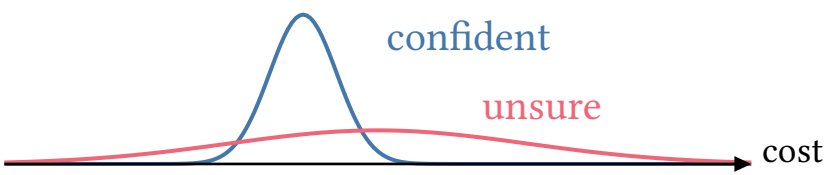
Finding Plans as Quickly as Possible

heuristic search under resource constraints

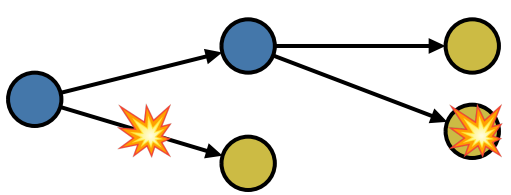
topology:



explore/exploit (Wissow & Asai, Outstanding Paper, ECAI-24):



- interested in: situated planning for a net-zero world



Overview

Kinds of Planning Problems
Popular Planning Method
Objectives and Constraints
Classical Planning
My Research
Wrap-Up

Much more to planning

- **task and motion planning**: robots moving themselves and objects
- **scheduling**: time, durative actions, shared resources
- **bidirectional search**: simultaneously from start and goal
- so much more!

Much more to planning

- **task and motion planning**: robots moving themselves and objects
- **scheduling**: time, durative actions, shared resources
- **bidirectional search**: simultaneously from start and goal
- so much more!

come to ICAPS and SoCS!

- short and long papers
- technical, position, application, demos
- really great communities
- good student funding

Deciding what to do?

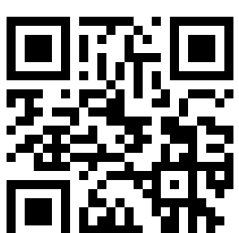
There is an actively-researched subfield of planning just for you!

Deciding what to do?

There is an actively-researched subfield of planning just for you!

Please reach out!

sjw@cs.unh.edu



looking for work!

(PhD expected May 2027)